

Day 1 — Vanilla RAG End to End

Disney-Focused Staff AI Engineer Notes

1. Core idea in simple words

RAG means Retrieval-Augmented Generation.

In simple words:

RAG is a way to make an LLM answer using your company's real documents instead of only using what the model already knows.

Think of an LLM like a very smart person who has general knowledge, but **does not automatically know Disney's latest internal policies, ad campaign rules, ticket refund rules, park schedules, or private engineering docs.**

So before asking the LLM to answer, we first **retrieve useful information** from trusted documents and give that information to the LLM.

Simple mental model

Without RAG:

```
User question
  ↓
LLM answers from memory
  ↓
May be outdated, generic, or wrong
```

With RAG:

```
User question
  ↓
Search trusted documents
  ↓
Pick relevant text
  ↓
Give text + question to LLM
  ↓
LLM answers using provided text
```

One-line definition

Vanilla RAG is the basic RAG pipeline: store documents, convert them into searchable chunks, retrieve relevant chunks for a user query, pass them to an LLM, and generate a grounded answer.

Here, **vanilla** means basic/simple version, without advanced agents, multi-step reasoning, query planning, graph retrieval, or complex orchestration.

2. Foundational concepts

2.1 What problem does RAG solve?

LLMs are powerful, but they have limitations.

For business systems, we often need answers based on:

- Latest company documents
- Private internal knowledge
- Product rules
- Customer policies
- Legal/compliance guidelines
- Engineering runbooks
- Logs, tickets, and operational documents
- Domain-specific knowledge

Example:

A Disney employee asks:

| “What is the refund policy for a cancelled park reservation during a weather event?”

The LLM may know general refund concepts, but it may not know Disney’s **current official policy**.

RAG solves this by searching internal policy documents first.

Question:

"What is the refund policy?"

Retrieve:

Official refund policy document

LLM answer:

Based on the retrieved policy

2.2 Why LLMs alone are not enough for real business systems

An **LLM**, or Large Language Model, is a model trained on huge amounts of text to predict and generate language.

But in production systems, LLMs alone are not enough because:

1. They may not know private data

The model does not automatically know your company’s internal documents.

Example:

Disney internal ad targeting policy

Disney vendor contract rules

Disney internal API documentation

These are not inside the model unless you provide them.

2. They may be outdated

The model may know older public information, but business rules change.

Example:

```
Old ticket cancellation policy
New ticket cancellation policy
```

If the LLM uses old knowledge, the answer can be wrong.

3. They may hallucinate

Hallucination means the LLM gives an answer that sounds confident but is not actually supported by facts.

Example:

```
User: What is the policy?
LLM: The refund window is 72 hours.
```

But maybe the real policy says 24 hours.

RAG reduces hallucination by forcing the model to answer using retrieved text.

4. They cannot cite sources by default

In enterprise systems, users often need to know:

```
Where did this answer come from?
Which document supports it?
Which section was used?
```

RAG can provide citations from retrieved documents.

5. They are not enough for audit/compliance

For Staff AI Engineer roles, this matters a lot.

In production, especially for enterprise or ad platforms, you need:

- Traceability
- Audit logs
- Source references
- Access control
- Data freshness
- Monitoring
- Evaluation

RAG helps build these controls around the LLM.

2.3 Pretraining vs prompting vs fine-tuning vs retrieval

These four terms are commonly confused.

Pretraining

Pretraining means the model is trained on a huge amount of general data before you use it.

Simple example:

The LLM learns language, facts, reasoning patterns, coding examples, general world knowledge.

You usually do not control pretraining.

Mental model

| Pretraining is like a person going to school for many years before joining your company.

Prompting

Prompting means giving instructions to the model at runtime.

Example:

You are a helpful assistant.
Answer in simple language.
Use bullet points.

Prompting changes how the model behaves, but it does not permanently teach new knowledge.

Mental model

| Prompting is like giving instructions before a task.

Fine-tuning

Fine-tuning means further training the model on specific examples so it behaves better for a certain task or style.

Example:

Train model on examples of Disney support replies.
Train model on internal question-answer style.
Train model to classify ad policy violations.

Fine-tuning is useful when you want the model to learn a pattern, style, or behavior.

But fine-tuning is not ideal for frequently changing knowledge.

Mental model

| Fine-tuning is like training an employee to follow your company's style and process.

Retrieval

Retrieval means searching external knowledge at runtime and giving it to the model.

Example:

```
Search the latest Disney policy document.  
Pass the matching policy section to the LLM.  
Ask the LLM to answer using that section.
```

Retrieval is best when knowledge changes often or must remain auditable.

Mental model

Retrieval is like giving the employee the latest company handbook before they answer.

Comparison

Method	What it does	Best for	Weakness
Pretraining	Gives general knowledge	Language, reasoning, general facts	Not private, may be outdated
Prompting	Gives runtime instructions	Style, format, behavior	Does not add reliable new knowledge
Fine-tuning	Trains model on examples	Repeated task behavior, tone, classification	Expensive, stale if facts change
Retrieval	Fetches external knowledge	Latest/private/auditable facts	Depends on retrieval quality

For most enterprise AI systems:

Use **RAG** for knowledge, use **prompting** for instruction, use **fine-tuning** only when behavior needs to improve repeatedly.

2.4 Basic concepts

Document

A **document** is any source of knowledge.

Examples:

```
PDF policy document  
HTML page  
Markdown file  
Confluence page  
Database record  
Customer support article  
Engineering runbook  
Ad policy guide
```

In RAG, documents are the raw knowledge source.

Chunk

A **chunk** is a smaller piece of a document.

Large documents are too big to search and pass directly to an LLM, so we split them.

Example document:

Disney Ad Policy Guide – 80 pages

Possible chunks:

Chunk 1: General ad eligibility
Chunk 2: Kids content advertising rules
Chunk 3: Alcohol-related advertising restrictions
Chunk 4: Regional compliance rules

Why chunking matters

If chunks are too large:

They contain too much unrelated text.

If chunks are too small:

They may lose context.

Good chunking is one of the most important RAG quality decisions.

Embedding

An **embedding** is a numeric representation of text.

Simple meaning:

| An embedding turns text into a list of numbers that captures meaning.

Example:

"refund policy for cancelled tickets"
→ [0.12, -0.44, 0.91, ...]

Text with similar meaning gets similar embeddings.

Example:

"refund policy"
"ticket cancellation reimbursement"

These may be close in embedding space because they are semantically similar.

Vector

A **vector** is a list of numbers.

An embedding is a vector.

Example:

```
[0.12, -0.44, 0.91, 0.03]
```

In real systems, embeddings may have hundreds or thousands of numbers.

Vector search

Vector search means searching for text by meaning, not exact words.

Example:

User asks:

```
"Can I get money back if my booking is cancelled?"
```

The document may say:

```
"Refunds are available for eligible cancelled reservations."
```

Keyword search may struggle because words differ.

Vector search can still match because the meaning is similar.

Retrieval

Retrieval means finding the most relevant chunks for a user query.

Example:

```
User question:
"What happens if a park reservation is cancelled?"

Retrieved chunks:
1. Refund policy
2. Cancellation policy
3. Weather exception policy
```

Reranking

Reranking means taking initially retrieved chunks and sorting them again using a stronger relevance model.

Simple example:

Initial vector search returns 20 chunks.

Reranker selects the best 5.

Vector search:
Fast but sometimes noisy

Reranker:
Slower but more accurate

Grounding

Grounding means forcing the answer to be based on provided evidence.

Example:

Answer only using the retrieved policy text.
If the answer is not present, say you do not know.

Grounding reduces hallucination.

Hallucination

Hallucination means the LLM generates unsupported or false information.

Example:

The LLM invents a policy deadline not present in the documents.

In RAG, hallucination can still happen if:

- Retrieved chunks are wrong
- Prompt is weak
- Context is polluted
- Model ignores evidence
- User asks something not covered

Context window

The **context window** is the amount of text the LLM can read in one request.

Example:

User question + retrieved chunks + system instructions

All of that must fit into the model's context window.

If context is too large:

- Cost increases
- Latency increases
- Model may get distracted
- Important details may be buried

2.5 Keyword search vs semantic search vs hybrid search

Keyword search

Keyword search matches exact or similar words.

Example query:

```
"refund cancellation ticket"
```

It finds documents containing those words.

Strength

Good for exact terms:

```
API names  
Policy IDs  
Error codes  
Product names  
Legal terms
```

Weakness

Bad when user uses different wording.

Example:

```
User says: "money back"  
Document says: "refund"
```

Keyword search may miss it.

Semantic search

Semantic search searches by meaning using embeddings.

Example:

```
"money back after cancellation"
```

Can match:

```
"refund eligibility for cancelled reservations"
```

Strength

Good for natural language questions.

Weakness

May retrieve text that feels related but is not exact enough.

Hybrid search

Hybrid search combines keyword search and semantic search.

This is often better in production.

Example:

Keyword search catches exact policy ID.
Semantic search catches meaning.
Hybrid combines both.

For enterprise RAG, hybrid search is often a strong default.

2.6 Recall and precision

These two terms are very important for RAG interviews.

Recall

Recall means:

| Did we find all the important relevant chunks?

High recall means the system does not miss important information.

Example:

User asks:

"What are the rules for ads targeting children?"

Relevant chunks:

Chunk A: Child-directed content rules
Chunk B: Regional child privacy rules
Chunk C: Prohibited targeting rules

If the system retrieves only Chunk A, recall is low.

Precision

Precision means:

| Of the chunks we retrieved, how many were actually useful?

High precision means retrieved context is clean and relevant.

Example:

Retrieved chunks:

Chunk A: Child-directed ads policy – useful
Chunk B: Disney dining reservation policy – not useful
Chunk C: Theme park parking policy – not useful

Precision is low because most retrieved chunks are noise.

Why both matter

If recall is low:

The LLM misses important facts.

If precision is low:

The LLM gets distracted by irrelevant facts.

A good RAG system needs both:

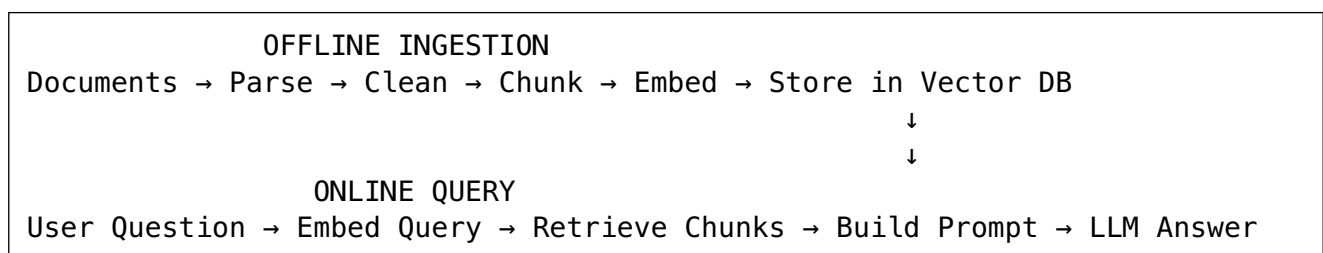
High recall + high precision = better answer quality

3. End-to-end Vanilla RAG flow

Vanilla RAG has two major pipelines:

1. **Offline ingestion pipeline**
2. **Online query pipeline**

3.1 Big picture



The offline pipeline prepares knowledge.

The online pipeline answers user questions.

3.2 Offline ingestion flow

Step 1: Data ingestion

Data ingestion means bringing source data into the RAG system.

Sources can be:

PDFs
Markdown files
Confluence pages
SharePoint documents
Internal APIs
Database rows
Support articles
Ad policy documents
Engineering runbooks

Disney example:

Source documents:
– Disney ad policy guide
– Campaign approval rules
– Content safety guidelines
– Park ticket refund policy
– Internal API documentation

Goal:

| Bring these documents into one pipeline so they can become searchable.

Step 2: Document parsing

Parsing means extracting usable text and structure from a document.

Example:

PDF input:

Page 1: Title
Page 2: Policy table
Page 3: Regional rules

Parsed output:

Title: Ad Policy Guide
Section: Regional Restrictions
Text: ...
Table: ...

Parsing must handle:

- Headings
- Paragraphs
- Tables
- Bullet points
- Page numbers
- Footnotes
- Images, if OCR is needed
- Code blocks, if technical docs

Why parsing matters

If parsing is bad, everything after it becomes bad.

Example bad parsing:

```
Refunds are  
not  
available  
unless...
```

The meaning may break.

Staff-level point:

| In production RAG, parsing quality is not a small detail. It directly controls answer quality.

Step 3: Cleaning and normalization

Cleaning means removing useless or noisy content.

Normalization means making text consistent.

Examples of cleaning:

```
Remove repeated headers  
Remove page footers  
Remove broken page numbers  
Remove navigation menus  
Remove duplicate boilerplate
```

Examples of normalization:

```
Convert smart quotes  
Fix whitespace  
Normalize date formats  
Normalize section titles  
Standardize product names
```

Bad text:

```
Disney Policy Guide | Page 21 | Confidential  
Disney Policy Guide | Page 21 | Confidential  
Refunds are available...
```

Clean text:

```
Refunds are available...
```

Why this matters

If noise is embedded, the vector search may retrieve noise.

Garbage in:

Bad document text

Garbage out:

Bad answer

Step 4: Chunking

Chunking means splitting cleaned documents into smaller pieces.

Example:

Full document:
"Disney Ad Policy Guide"

Chunks:

1. General rules
2. Child-directed content rules
3. Regional restrictions
4. Prohibited claims
5. Review workflow

Common chunking approaches

Fixed-size chunking

Split every fixed number of words or tokens.

Example:

Every 500 tokens with 50 token overlap

Simple but may cut sections awkwardly.

Section-based chunking

Split based on headings.

Example:

Section 3.1: Refund eligibility
Section 3.2: Cancellation deadlines
Section 3.3: Exceptions

Usually better for policy and documentation.

Semantic chunking

Split based on meaning.

Example:

Keep related paragraphs together even if size varies.

Better quality, but more complex.

Step 5: Chunk size and overlap trade-offs

Chunk size

Chunk size means how much text goes into one chunk.

Small chunk example:

"Refunds are available for eligible cancellations."

Large chunk example:

Full refund policy section with eligibility, exceptions, examples, deadlines.

Small chunks

Pros:

- More precise
- Less irrelevant text
- Lower token cost per chunk

Cons:

- May miss surrounding context
 - Can break meaning
 - More chunks to manage
-

Large chunks

Pros:

- More context
- Better for complex policies
- Less risk of splitting important details

Cons:

- More noise
 - Higher token cost
 - Lower precision
 - May exceed context budget
-

Overlap

Overlap means repeating some text between neighboring chunks.

Example:

Chunk 1:
Paragraphs 1–5

Chunk 2:
Paragraphs 5–9

Paragraph 5 overlaps.

Why overlap helps

It avoids losing context at chunk boundaries.

Trade-off

More overlap means:

More storage
More duplicate retrieval
More embedding cost

Simple starting point:

Chunk size: 300–800 tokens
Overlap: 10–20%

But the right value depends on your documents.

Step 6: Embedding generation

Embedding generation means converting each chunk into a vector.

Example:

Chunk:
"Refunds are available for eligible cancelled reservations."

Embedding:
[0.15, -0.32, 0.78, ...]

You use an **embedding model** for this.

An embedding model is a model that converts text into vectors.

Why embeddings matter

The embedding decides what “similar meaning” means.

If the embedding model is weak, retrieval will be weak.

Example:

A good embedding model understands:


```
"refund"  
"money back"  
"reimbursement"
```

are related.

Step 7: Vector indexing

Vector indexing means storing embeddings in a way that makes similarity search fast.

Without an index:

```
Compare query vector with every chunk vector
```

This is slow at scale.

With an index:

```
Quickly find nearby vectors
```

Common vector databases:

```
Pinecone  
Qdrant  
Weaviate  
Milvus  
Chroma  
pgvector  
OpenSearch vector search
```

Staff-level point

For small systems, simple vector search is fine.

For large Disney-scale systems, you care about:

- Latency
 - Recall
 - Index memory
 - Multi-tenant separation
 - Metadata filtering
 - Freshness
 - Re-indexing strategy
 - Cost
-

Step 8: Metadata storage

Metadata means extra information stored with each chunk.

Example chunk text:

Refunds are available for eligible cancellations.

Metadata:

```
{
  "document_id": "refund_policy_2026",
  "title": "Ticket Refund Policy",
  "section": "Weather Exceptions",
  "version": "2026-07",
  "region": "US",
  "access_level": "internal",
  "source_url": "...",
  "created_at": "2026-07-01",
  "tenant_id": "parks-business-unit"
}
```

Why metadata is important

Metadata helps with:

- Filtering
- Security
- Citations
- Freshness
- Debugging
- Evaluation
- Tenant isolation

Example:

User is from India business unit.

You should retrieve only:

```
region = India
tenant_id = correct team
access_level allowed for user
```

Missing metadata is a major production problem.

3.3 Online retrieval flow

Now the user asks a question.

Example:

```
"Can we show personalized ads to children on Disney+?"
```

Step 1: Receive user query

The backend receives:

```
{
  "user_id": "u123",
  "tenant_id": "ads-platform",
  "question": "Can we show personalized ads to children on Disney+?"
}
```

At this stage, production systems may also check:

- Authentication
- Authorization
- Tenant access
- Rate limits
- Query safety
- Logging

Step 2: Query embedding

The user question is converted into an embedding.

```
Question:
"Can we show personalized ads to children on Disney+?"

Embedding:
[0.22, -0.17, 0.89, ...]
```

Now the system can compare the query vector with chunk vectors.

Step 3: Metadata filtering

Before or during search, we can filter by metadata.

Example:

```
{
  "tenant_id": "ads-platform",
  "region": "US",
  "document_status": "active",
  "access_level": "allowed"
}
```

This avoids retrieving irrelevant or unauthorized documents.

Staff-level point:

Metadata filtering is not optional in enterprise systems. It is required for correctness, security, and compliance.

Step 4: Top-k retrieval

Top-k retrieval means retrieving the top k most similar chunks.

If $k = 5$, retrieve 5 chunks.

Example:

```
Top 5 chunks:
1. Children advertising policy
2. Personalized targeting restrictions
3. Disney+ profile age rules
4. Regional privacy compliance
5. Ad review escalation process
```

Wrong top-k problem

If k is too small:

```
You may miss important context.
```

If k is too large:

```
You may add irrelevant context.
Cost and latency increase.
```

Common starting values:

```
top_k = 5 to 10
```

But it should be tuned using evaluation.

Step 5: Reranking

Initial vector retrieval is fast but may not be perfect.

A **reranker** takes the query and candidate chunks, then sorts them by stronger relevance.

Example:

```
Initial retrieval:
20 chunks

Reranker:
Select best 5 chunks
```

Before reranking:

```
Chunk A: Related to children content
Chunk B: Exact policy about child-targeted ads
Chunk C: Related to Disney+ profiles
```

After reranking:

```
1. Exact child-targeted ads policy
2. Personalized targeting restrictions
3. Regional privacy compliance
```

Reranking usually improves precision.

Step 6: Context assembly

Context assembly means preparing retrieved chunks to send to the LLM.

Example:

```
Context:
[Source 1]
Title: Disney Ad Policy
Section: Children and Personalized Ads
Text: ...

[Source 2]
Title: Privacy Compliance Guide
Section: Child Data
Text: ...
```

Good context assembly includes:

- Clean chunk text
- Source title
- Section name
- Version
- Citation ID
- Only necessary chunks
- No duplicate content
- Clear separation between sources

Bad context assembly:

```
Random chunks pasted together with no source labels.
```

That makes citations and grounding harder.

Step 7: Prompt construction for RAG

Prompt construction means building the final instruction sent to the LLM.

A good RAG prompt usually includes:

1. Role
2. Rules
3. Retrieved context
4. User question
5. Output format
6. Instruction to avoid unsupported claims

Example:

You are a Disney internal policy assistant.

Answer the user question using only the provided context.

If the answer is not present, say: "I could not find this in the provided documents."

Cite the source IDs used.

Context:

[doc_1_section_3]

...

Question:

Can we show personalized ads to children on Disney+?

Answer:

Important rule

Do not simply say:

Answer the question.

Better:

Answer only using the provided sources.

Do not invent policy details.

Cite every major claim.

Step 8: Answer generation

The LLM generates the final answer using the prompt and context.

Good answer:

No, personalized ads cannot be shown to children if the policy prohibits behavior-based targeting for child profiles. The relevant policy section states...

Source: Disney Ad Policy, Section 3.2

Bad answer:

Yes, it should be fine as long as parents agree.

The bad answer may be unsupported or too generic.

Step 9: Citation-aware answering

Citation-aware answering means the answer includes sources.

Example:

Personalized ads are not allowed for child profiles under the child-directed content policy. [Source: Ad Policy Guide, Section 4.2]

Citations help with:

- Trust
- Auditability
- Debugging
- Compliance
- User confidence

Staff-level point:

| In enterprise RAG, citations are not decoration. They are part of the reliability contract.

Step 10: Feedback loop

A **feedback loop** means collecting signals to improve the system.

Feedback can include:

User thumbs up/down
User says answer was wrong
No answer found
Low confidence response
Clicked citation
Human reviewer correction
Retrieved chunks used by the model
Latency/cost logs

This helps improve:

- Chunking
 - Retrieval
 - Prompting
 - Ranking
 - Document freshness
 - Evaluation datasets
-

3.4 Vanilla RAG pseudocode

Simple backend-style pseudocode:

```
# Offline ingestion
documents = load_documents()
parsed_docs = parse_documents(documents)
clean_docs = clean_text(parsed_docs)
chunks = split_into_chunks(clean_docs)

for chunk in chunks:
```

```

embedding = embedding_model.embed(chunk.text)
vector_db.insert(
    vector=embedding,
    text=chunk.text,
    metadata=chunk.metadata
)

# Online query
def answer_question(user_question, user_context):
    query_vector = embedding_model.embed(user_question)

    candidate_chunks = vector_db.search(
        vector=query_vector,
        top_k=10,
        filters={
            "tenant_id": user_context.tenant_id,
            "access_level": user_context.access_level,
            "status": "active"
        }
    )

    reranked_chunks = reranker.rank(
        query=user_question,
        chunks=candidate_chunks
    )

    context = build_context(reranked_chunks[:5])

    prompt = build_rag_prompt(
        question=user_question,
        context=context
    )

    answer = llm.generate(prompt)

    return answer

```

This is vanilla RAG.

4. Inter-relation between all stages

RAG is a chain. Each stage affects the next stage.

```

Parsing → Cleaning → Chunking → Embedding → Retrieval → Reranking → Prompt
→ Answer

```

If one stage is weak, the final answer becomes weak.

4.1 How chunking affects embeddings

An embedding represents the meaning of a chunk.

If the chunk is clean and focused, the embedding is clear.

Good chunk:

Refund eligibility for weather-related park cancellations.

Embedding meaning:

Refund + weather + cancellation

Bad chunk:

Refund policy + parking rules + restaurant booking + footer text

Embedding meaning becomes mixed.

The vector search may not know what this chunk is really about.

Key idea

| A chunk should usually represent one main idea.

4.2 How embeddings affect retrieval quality

If embeddings capture meaning well, retrieval improves.

Example query:

"Can guests get money back if the park closes due to weather?"

Good embedding retrieves:

Weather cancellation refund policy

Bad embedding retrieves:

Weather forecast page
Park operating hours
General cancellation FAQ

Embedding quality directly controls retrieval quality.

4.3 How retrieval quality affects final answer quality

The LLM can only answer well if it receives the right context.

Good retrieval:

Question + correct policy chunks

Likely good answer.

Bad retrieval:

Question + irrelevant chunks

Likely bad answer.

Important point:

| RAG does not magically fix bad retrieval. If retrieval fails, the LLM may still hallucinate.

4.4 How context size affects cost, latency, and answer quality

Context size means how much text you send to the LLM.

More context is not always better.

More context can improve recall

If you include more chunks, you may include the right answer.

But more context can hurt precision

Too many chunks can confuse the model.

Example:

Relevant chunk:

Child ads are restricted.

Irrelevant chunk:

General advertising creative guidelines.

Another irrelevant chunk:

Adult audience campaign policy.

The LLM may mix rules incorrectly.

More context also increases:

- Token cost
- Latency
- Memory usage
- Risk of distraction
- Prompt complexity

Staff-level thinking:

| The goal is not maximum context. The goal is minimum sufficient context.

4.5 How poor ingestion can break the full pipeline

Poor ingestion is one of the biggest RAG failures.

Example:

Original document:

```
Refunds are not available after the event starts.
```

Bad parsing:

```
Refunds are available after the event starts.
```

Now the system may answer incorrectly.

Another example:

Original section title:

```
Child-directed advertising restrictions
```

Parser loses heading.

Chunk only says:

```
Restrictions apply in certain cases.
```

Now the chunk has poor meaning.

Key idea

| Most RAG quality problems start before retrieval, during ingestion.

5. Production-grade challenges

Now let's discuss common real-world RAG problems.

5.1 Bad chunking choices

Bad chunking can cause:

- Missing context
- Mixed topics
- Duplicate retrieval
- Wrong answers
- High token cost

Example:

Chunk 1:
"Personalized ads are allowed..."
Chunk 2:
"...except for child profiles."

If the system retrieves only Chunk 1, the answer becomes dangerous.

Correct chunk should include the full rule:

Personalized ads are allowed for eligible adult profiles but prohibited for child profiles.

5.2 Missing metadata

Without metadata, you cannot filter correctly.

Example missing metadata:

No region
No version
No tenant
No access level
No document status

Problem:

User asks from US team.
System retrieves outdated EU document.

Or:

User from one business unit sees another unit's private document.

This is not just a quality issue. It is a security issue.

5.3 Stale data

Stale data means outdated data.

Example:

Old policy from 2024
New policy from 2026

If both exist in the vector database, the system may retrieve the old one.

Solutions:

- Version metadata
- Active/inactive document status
- Scheduled re-indexing
- Source sync tracking

- Last updated timestamps
 - Delete or archive old embeddings
-

5.4 Duplicate documents

Duplicate documents create duplicate chunks.

Problem:

Same policy appears 5 times.

Retrieval returns the same idea repeatedly.

This wastes context and hides other important chunks.

Solution:

- Document deduplication
 - Chunk-level deduplication
 - Hashing
 - Canonical source selection
 - Version management
-

5.5 Poor parsing quality

Common parsing problems:

- Tables become broken text
- Headers are lost
- Footers repeat everywhere
- Bullet hierarchy is lost
- Columns are mixed incorrectly
- OCR errors corrupt text

Example:

Original table:

Profile type	Personalized ads allowed?
Child	No
Adult	Yes

Bad parse:

Child Adult No Yes

Now meaning is unclear.

For policy documents, table parsing is very important.

5.6 Low recall

Low recall means the system misses relevant chunks.

Causes:

- Poor chunking
- Weak embedding model
- Wrong metadata filter
- Too small top-k
- Query wording mismatch
- Missing documents
- Bad indexing

Impact:

The model does not receive enough evidence.

5.7 Low precision

Low precision means retrieved chunks are noisy.

Causes:

- Chunk too large
- Top-k too high
- Weak embeddings
- No reranking
- Missing filters
- Duplicate documents
- Broad query

Impact:

The model gets distracted.

5.8 Wrong top-k

If top_k is too low:

Important evidence may be missed.

If top_k is too high:

Too much irrelevant text enters context.

The right top-k depends on:

- Document type
- Query complexity
- Embedding model

- Reranker quality
 - Context window size
 - Cost target
 - Latency target
-

5.9 Context pollution

Context pollution means irrelevant or conflicting text is included in the prompt.

Example:

```
Chunk 1: Current child ads policy
Chunk 2: Old child ads policy
Chunk 3: Adult ads policy
Chunk 4: Unrelated campaign policy
```

The LLM may mix them.

Context pollution often causes hallucination.

5.10 Hallucinated answers even when retrieval exists

RAG reduces hallucination, but does not eliminate it.

Hallucination can happen when:

- Retrieved context is irrelevant
- Context is incomplete
- Prompt allows guessing
- LLM uses prior knowledge instead of sources
- User asks outside available documents
- Documents conflict
- Answer requires reasoning across many chunks

A good RAG system must support:

```
"I could not find enough information in the provided sources."
```

5.11 Slow retrieval

Retrieval can become slow because of:

- Large index
- Poor index configuration
- No filtering
- Slow metadata store
- Network latency
- Reranker latency
- Multiple retrieval calls
- Large top-k

Production target often needs:

p95 latency

p95 latency means 95% of requests finish under a certain time.

Example:

p95 answer latency < 3 seconds

5.12 High token cost

Tokens are pieces of text processed by the LLM.

More retrieved context means more tokens.

High token cost comes from:

- Too many chunks
- Large chunks
- Duplicate chunks
- Long prompts
- Long answers
- Uncompressed context

Staff-level goal:

Use enough context to answer correctly, but not more.

5.13 Multi-tenant isolation concerns

Multi-tenant means one system serves multiple teams, customers, or business units.

Example:

Disney Ads team
Disney Parks team
Disney Streaming team
Disney internal HR team

Each tenant may have different documents and permissions.

RAG must prevent:

Tenant A seeing Tenant B's documents

This requires:

- Tenant metadata
- Access filters
- Authorization checks
- Separate indexes or namespaces

- Audit logs
 - Secure document ingestion
-

5.14 Security and privacy concerns

RAG systems may handle sensitive data.

Risks:

- Unauthorized document retrieval
- Prompt injection inside documents
- PII leakage
- Logging sensitive queries
- Exposing hidden system prompts
- Cross-tenant data leakage
- Unsafe tool access

PII means personally identifiable information.

Examples:

Email
Phone number
Address
Customer ID
Payment details

Staff-level point:

RAG security is not only about the LLM. It starts from ingestion, metadata, retrieval filters, logs, and access control.
--

5.15 Monitoring blind spots

Monitoring means observing system behavior in production.

RAG monitoring should track:

- Retrieval latency
- LLM latency
- Total latency
- Token usage
- Retrieved document IDs
- Top-k distribution
- No-answer rate
- User feedback
- Citation coverage
- Error rate
- Cost per request

Blind spot example:

You track only final LLM response latency, but not retrieval quality.

Then you cannot know why answers are bad.

5.16 Evaluation blind spots

Evaluation means measuring quality.

For RAG, you need to evaluate:

- Did retrieval find correct chunks?
- Did answer use retrieved chunks?
- Was the answer faithful?
- Were citations correct?
- Did the model refuse when answer was missing?
- Did latency stay acceptable?
- Did cost stay acceptable?

Bad evaluation:

Only checking if answer sounds good.

Good evaluation:

Checking retrieval + grounding + answer correctness + citation quality.

6. Optimization strategies

Now let's discuss how to improve a vanilla RAG system.

6.1 Better chunking strategies

Improve chunking by using document structure.

For policy docs:

Use section-based chunking.
Keep heading + body together.
Keep exceptions with the rule.

For technical docs:

Keep code block with explanation.
Keep API endpoint with request/response examples.

For FAQs:

Keep question and answer together.

Good chunk should include:

- Main heading
- Subheading
- Complete idea
- Enough local context
- Source metadata

Example good chunk:

```
Title: Disney Ad Policy
Section: Child Profiles
Text:
Personalized advertising is not allowed for child profiles...
```

6.2 Metadata filtering

Use metadata to reduce search space.

Example filters:

```
{
  "tenant_id": "ads-platform",
  "region": "US",
  "document_status": "active",
  "access_level": "internal",
  "product": "Disney+"
}
```

Benefits:

- Better precision
- Lower latency
- Better security
- Less context pollution

6.3 Hybrid search

Use both keyword and semantic search.

Example:

User asks:

```
"What is policy AP-213 for kids ads?"
```

Keyword search finds:

```
AP-213
```

Semantic search finds:

kids advertising restriction

Hybrid search combines both.

This is useful when queries include:

- Policy IDs
- Error codes
- API names
- Product names
- Natural language descriptions

6.4 Query rewriting

Query rewriting means improving the user query before retrieval.

User query:

"Can we do this for kids?"

Rewritten query:

"Disney+ child profile personalized advertising policy and restrictions"

Why this helps:

- User queries are often vague
- Better query improves retrieval
- Synonyms can be added

But be careful.

Bad query rewriting can change user intent.

6.5 Reranking

Use reranking to improve final chunk selection.

Typical flow:

Vector search top 30
↓
Reranker selects top 5
↓
Send top 5 to LLM

Benefits:

- Better precision
- Less noise
- Better final answer

Trade-off:

- More latency
- More cost
- Extra model dependency

6.6 Context compression

Context compression means reducing retrieved content before sending it to the LLM.

Example:

Original chunk:

1000 tokens

Compressed context:

300 tokens focused on the user question

Ways to compress:

- Remove irrelevant sentences
- Summarize retrieved chunks
- Extract only matching sections
- Deduplicate repeated text

Trade-off:

Compression can accidentally remove important details.

6.7 Better prompt construction

A strong RAG prompt should say:

Use only the provided context.
Cite sources.
Do not guess.
If context is insufficient, say so.
Separate facts from assumptions.
Prefer current active policy over old policy.

Example prompt skeleton:

You are an internal Disney AI assistant.

Rules:

1. Answer only using the provided context.
2. If the answer is not present, say you do not have enough information.
3. Cite every major claim.
4. Do not use outside knowledge for policy decisions.
5. If sources conflict, mention the conflict.

Context:

{retrieved_context}

Question:

{user_question}

Answer:

6.8 Better top-k selection

Instead of fixed `top_k = 5` for everything, adapt it.

Simple question:

"What is the refund deadline?"

Maybe top-k 3 is enough.

Complex question:

"Compare child advertising rules across US and EU."

Maybe top-k 10 or more is needed.

Adaptive top-k can use:

- Query complexity
- Similarity scores
- Metadata filters
- Reranker confidence
- Context budget

6.9 Retrieval caching

Caching means storing results so repeated requests are faster.

Cache examples:

Same query embedding
Same retrieval result
Same final answer for public FAQs

Benefits:

- Lower latency
- Lower cost
- Less load on vector database

Risks:

- Stale answers
- Permission leakage if cache key ignores user access
- Wrong answer reused for a different tenant

Safe cache key should include:

```
query
tenant_id
user_access_level
document_version
filters
```

6.10 Embedding model selection trade-offs

Choosing an embedding model affects quality, cost, and latency.

Consider:

- Accuracy
- Domain understanding
- Language support
- Embedding dimension size
- Cost
- Latency
- Self-hosted vs API
- Compliance requirements

For Disney-scale enterprise systems, ask:

```
Does it understand policy language?
Does it support our languages?
Can we use it with private data?
What is the latency?
What is the cost per million chunks?
How often do we need to re-embed?
```

6.11 Index tuning basics

A vector index has configuration choices.

Simple idea:

Better accuracy may cost more latency and memory.
Faster search may reduce recall.

Important tuning questions:

- How many vectors?
- What latency target?
- What recall target?
- How often data changes?
- Do we need metadata filtering?
- Do we need separate indexes per tenant?
- Can we rebuild index safely?
- What happens during re-indexing?

Staff-level thinking:

| Index tuning is a trade-off between recall, latency, memory, freshness, and operational complexity.

6.12 Freshness strategies

Freshness means keeping RAG knowledge up to date.

Strategies:

Scheduled sync

Re-index documents every night.

Good for documents that change slowly.

Event-based sync

When a policy changes, trigger re-indexing.

Better for frequently updated systems.

Versioned documents

```
policy_v2026_07 active=true  
policy_v2025_12 active=false
```

Prevents old policies from being used.

Soft delete old chunks

Mark old chunks inactive instead of immediately deleting.


```
{
  "active": false,
  "replaced_by": "policy_v2026_07"
}
```

Useful for audit.

6.13 Cost vs quality vs latency trade-off

This is a core Staff-level trade-off.

Better quality may require:

- More chunks
- Reranking
- Better embedding model
- Larger LLM
- More prompt instructions
- More evaluation

But this may increase:

- Cost
- Latency
- Complexity

Lower latency may require:

- Smaller top-k
- Caching
- Faster vector DB
- Smaller model
- Fewer reranking steps

But this may reduce:

- Recall
- Answer quality
- Citation quality

Lower cost may require:

- Smaller chunks
- Context compression
- Cheaper models
- Caching
- Shorter answers

But this may reduce:

- Depth
- Accuracy
- User trust

Staff-level answer:

I would define target quality, latency, and cost SLOs first, then tune retrieval, reranking, context size, and model choice against an evaluation dataset.

6.14 When vanilla RAG is enough

Vanilla RAG is enough when:

- Questions are simple
- Documents are well-structured
- One retrieval step is enough
- Answers are mostly extractive
- Few tools are needed
- Data is not highly relational
- Latency requirements are strict
- System needs to be simple and maintainable

Example:

```
Internal FAQ assistant
Policy Q&A assistant
Simple engineering documentation bot
Support article search assistant
```

6.15 When advanced RAG is needed

Advanced RAG is needed when:

- Query needs multiple steps
- User question is vague
- Information is spread across many documents
- Documents conflict
- Need SQL/API/tool calls
- Need reasoning over structured and unstructured data
- Need planning
- Need memory/state
- Need human approval
- Need graph relationships
- Need complex evaluation

Example:

```
"Compare current ad targeting policy against this proposed campaign and
tell me what approvals are needed."
```

This may require:

- Retrieve policy documents
- Retrieve campaign metadata
- Check region

- Check audience age group
- Call approval workflow API
- Generate decision
- Route to human reviewer

That is beyond vanilla RAG.

7. Easy real-world example

Let's build a simple Disney example.

Use case

Disney has an internal assistant for the Ads Platform team.

User asks:

"Can we run personalized ads for child profiles on Disney+?"

Offline ingestion

Source documents

1. Disney+ Advertising Policy
 2. Child Privacy Compliance Guide
 3. Regional Ad Targeting Rules
 4. Campaign Review Workflow

Parse

Extract text from PDFs and internal docs.

Section: Child Profiles
Section: Personalized Advertising
Section: Compliance Exceptions

Clean

Remove:

Page numbers
Repeated footer
Navigation text
Old draft comments

Chunk

Good chunk:

Title: Disney+ Advertising Policy
Section: Child Profiles and Personalized Ads

Text:
Personalized ads are not allowed for child profiles...

Embed

Convert chunk into vector.

"Personalized ads are not allowed for child profiles..."
→ vector

Store

Store in vector database with metadata:

```
{  
  "product": "Disney+",  
  "policy_area": "advertising",  
  "audience": "child",  
  "region": "US",  
  "version": "2026-07",  
  "status": "active"  
}
```

Online query

User query

Can we run personalized ads for child profiles on Disney+?

Query embedding

Convert question to vector.

Retrieve

Find similar chunks.

Retrieved:

Chunk 1: Child profile ad restrictions
Chunk 2: Personalized targeting rules
Chunk 3: Child privacy compliance

Rerank

Best chunks selected:

1. Child profile ad restrictions
2. Personalized targeting rules

Prompt

Answer only using the context.
Cite sources.
If the policy is not present, say you do not know.

Answer

No. Based on the retrieved policy, personalized ads are not allowed for child profiles on Disney+. The relevant policy section says child profiles cannot be targeted using personalized advertising signals.

Sources:

- Disney+ Advertising Policy, Child Profiles section
- Child Privacy Compliance Guide, Personalized Targeting section

8. Staff-level interview angle

8.1 How to explain RAG in a system design interview

A strong simple explanation:

RAG is a pattern where we connect an LLM to an external knowledge base. Instead of relying only on the model's pretrained knowledge, we ingest trusted documents, split them into chunks, create embeddings, store them in a vector index, retrieve relevant chunks for each user query, and pass those chunks to the LLM so it can generate a grounded answer with citations.

Then add:

In production, the hard parts are not just calling the LLM. The hard parts are ingestion quality, chunking, metadata, retrieval precision and recall, freshness, security, evaluation, latency, and cost.

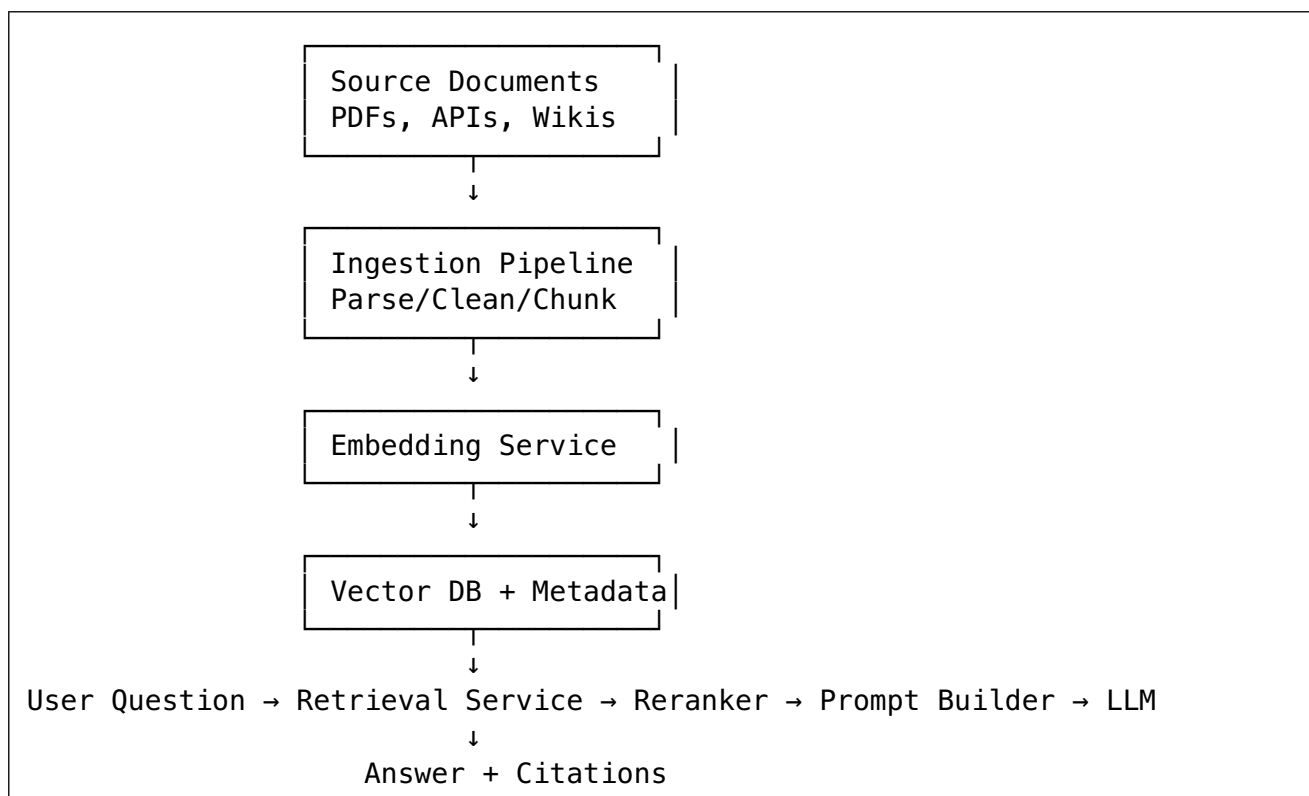
That sounds Staff-level.

8.2 How to discuss the architecture

Use this structure:

1. Ingestion pipeline
2. Storage/indexing layer
3. Query/retrieval service
4. Reranking/context assembly
5. LLM generation service
6. Citation and response layer
7. Evaluation/monitoring layer
8. Security and governance layer

Simple architecture:



8.3 How to discuss failure modes

In an interview, do not say only:

RAG gives better answers.

Say:

RAG can still fail if the wrong documents are retrieved, chunks are poorly created, metadata filters are missing, data is stale, or the model ignores the context. So I would evaluate both retrieval quality and answer quality separately.

Important failure modes:

Wrong chunk retrieved
Important chunk missed
Old policy retrieved
Unauthorized document retrieved
Conflicting sources
LLM invents unsupported answer
Citations point to irrelevant source
Latency too high
Token cost too high

8.4 How to discuss trade-offs

Chunk size trade-off

Small chunks improve precision but may lose context.
Large chunks preserve context but may add noise and cost.

Top-k trade-off

Lower top-k reduces cost and latency but may miss evidence.
Higher top-k improves recall but may pollute context.

Reranking trade-off

Reranking improves relevance but adds latency and cost.

Hybrid search trade-off

Hybrid search improves recall for exact terms and semantic queries, but adds complexity.

Freshness trade-off

Frequent re-indexing improves freshness but increases compute cost and operational complexity.

Security trade-off

Shared indexes are easier to operate, but strict tenant isolation may require namespaces, filters, or separate indexes.

8.5 What a Staff AI Engineer should own

A Staff AI Engineer should not only build a demo.

They should own the production quality of the system.

Key ownership areas:

1. Architecture

Design the full RAG backend:

Ingestion
Indexing
Retrieval
Generation
Evaluation
Monitoring
Security
Deployment

2. Quality

Define how quality is measured.

Examples:

```
Retrieval recall
Retrieval precision
Faithfulness
Citation accuracy
Answer correctness
No-answer accuracy
```

3. Reliability

Make the system stable.

Examples:

```
Retry failed ingestion
Handle vector DB timeout
Fallback when reranker fails
Return safe answer when context is missing
```

4. Scalability

Prepare for growth.

Examples:

```
Millions of chunks
Many tenants
High QPS
Large document updates
Low latency requirements
```

5. Security

Protect data.

Examples:

```
Access control
Tenant isolation
PII handling
Audit logs
Secure metadata filtering
```

6. Cost control

Optimize token and infrastructure cost.

Examples:

Caching
Context compression
Right top-k
Model routing
Batch embedding

7. Observability

Make the system debuggable.

Track:

Which chunks were retrieved?
Which source was cited?
What was the latency?
What was the token cost?
Did user like the answer?
Was the answer grounded?

8.6 How RAG fits into Disney AI-powered systems

Example 1: Disney ad platform assistant

Use RAG to answer:

Can this campaign target this audience?
Which policy applies?
What approval workflow is needed?
Is this creative allowed?

Documents:

Ad policies
Regional compliance rules
Brand safety rules
Campaign review guidelines

Example 2: Disney enterprise AI assistant

Use RAG to answer:

How do I onboard a new service?
What is the deployment checklist?
Which API should I use?
What is the incident response process?

Documents:

Engineering runbooks
Architecture docs
API docs
Security policies
SRE playbooks

Example 3: Disney customer support assistant

Use RAG to answer:

What is the refund rule?
What is the cancellation policy?
What are park reservation conditions?

Documents:

Support articles
Ticket policies
Park operation rules
Guest service procedures

Final revision summary

Vanilla RAG is simple in concept:

Retrieve relevant knowledge → Give it to LLM → Generate grounded answer

But production RAG is hard because every stage matters:

Bad parsing → bad chunks
Bad chunks → bad embeddings
Bad embeddings → bad retrieval
Bad retrieval → bad context
Bad context → bad answer
Bad answer → low trust

A Staff AI Engineer should think beyond the demo:

Quality
Latency
Cost
Security
Freshness
Evaluation
Monitoring
Scalability

The most important mental model

| RAG quality is not mainly an LLM problem. It is a full backend system quality problem.

For a Disney Staff AI Engineer interview, your strongest message should be:

I would design RAG as a reliable knowledge-backed AI service, with strong ingestion, metadata, retrieval evaluation, citation-aware generation, access control, observability, and continuous improvement loops.